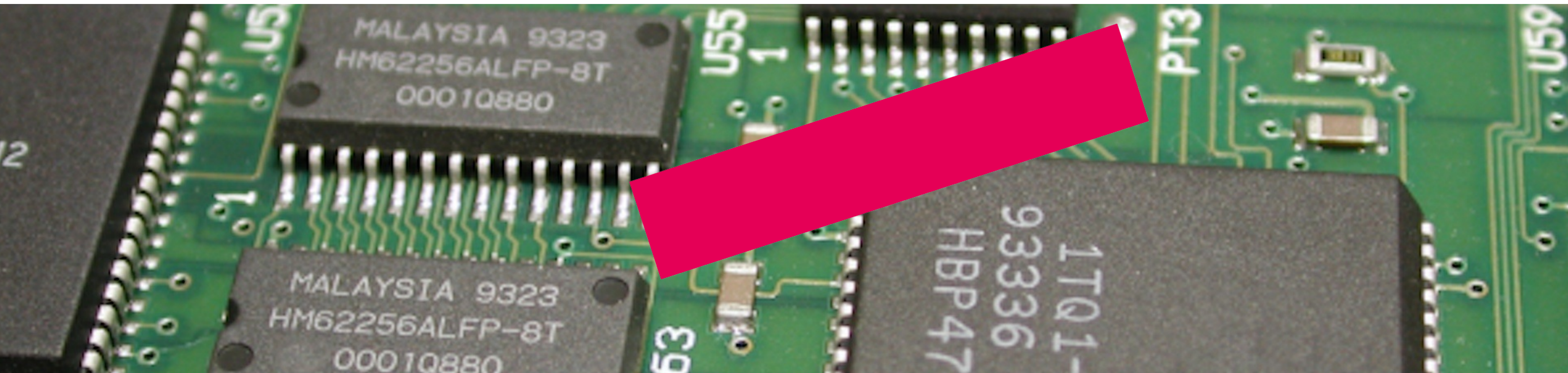


Prog 5 - 4. Interfaces and abstract classes



Electrical Engineering / Embedded Systems
Faculty of Engineering

Johan.Korten@han.nl

To begin with...

Attendance

Schedule (exact info see #00 and roster at insite.han.nl)

Week (Thu)	Design principle	C++	UML	BME280 library step	Workshop (Fri)
Step 1 - 3 Sep	Single Responsibility	Scope, namespaces, string	Use cases / class diagram	-	4 Sep: Git
Step 2 - 10 Sep	Open-Closed Principle	Constructors, iterators, lambdas	Inheritance / generalization	Weather-station use cases	11 Sep: Requirements
Step 3 - 17 Sep	Liskov Substitution Principle	Lists, inline functions, default params	Activity diagrams	Core + Bus + Arduinol2cBus	-
Step 4 - 24 Sep	Interface Segregation Principle	Interfaces and abstract classes	Dependencies	Linuxl2cBus on the Pi; Bus / Clock split	-
Step 5 - 1 Oct	Dependency Inversion Principle	Threads, callbacks	Sequence diagrams	Sampler thread + callback; MQTT Publisher	-
Step 6 - 8 Oct	Coupling and cohesion	Polymorphism	Composition, packages	Polymorphic sensors; UML of the library	-
Step 7 - 15 Oct	Embedded SOLID	Lifecycle: constructors, destructors, const	-	RAII; host unit tests with MockBus; refactor	-

Note: subject to changes as we go...

Interfaces / Abstract classes. To start with: some terms

A **virtual function in C++** is a member function in a class that we declare within the base class and redefine in a derived class.

A **pure virtual function in C++** is nothing but a virtual function that we know exists but cannot be implemented. It is simply declared, not implemented.

Implementing a **pure virtual function in C++**

```
/* 0 does not indicate that we initialize the function to a null or zero value */  
virtual void function() = 0;
```

Last week you wrote this: the Bus interface

```
class Bus {
public:
    virtual ~Bus() = default;

    // Read `len` bytes starting at register `reg`.
    virtual bool read(uint8_t reg, uint8_t* data, size_t len) = 0;

    // Write `len` bytes starting at register `reg`.
    virtual bool write(uint8_t reg, const uint8_t* data, size_t len) = 0;

    // Block for at least `us` microseconds.
    virtual void delayUs(uint32_t us) = 0;
};

// Only <stdint> and <stddef>. No Wire.h, no Linux, no Arduino.
```

An interface is a contract: platform 1, Arduino

```
bool ArduinoI2cBus::read(uint8_t reg, uint8_t* data, size_t len) {  
    wire_.beginTransmission(address_);  
    wire_.write(reg);  
    if (wire_.endTransmission(false) != 0) return false;    // repeated start  
    if (wire_.requestFrom(address_, len) != len) return false;  
    for (size_t i = 0; i < len; ++i) data[i] = wire_.read();  
    return true;  
}
```

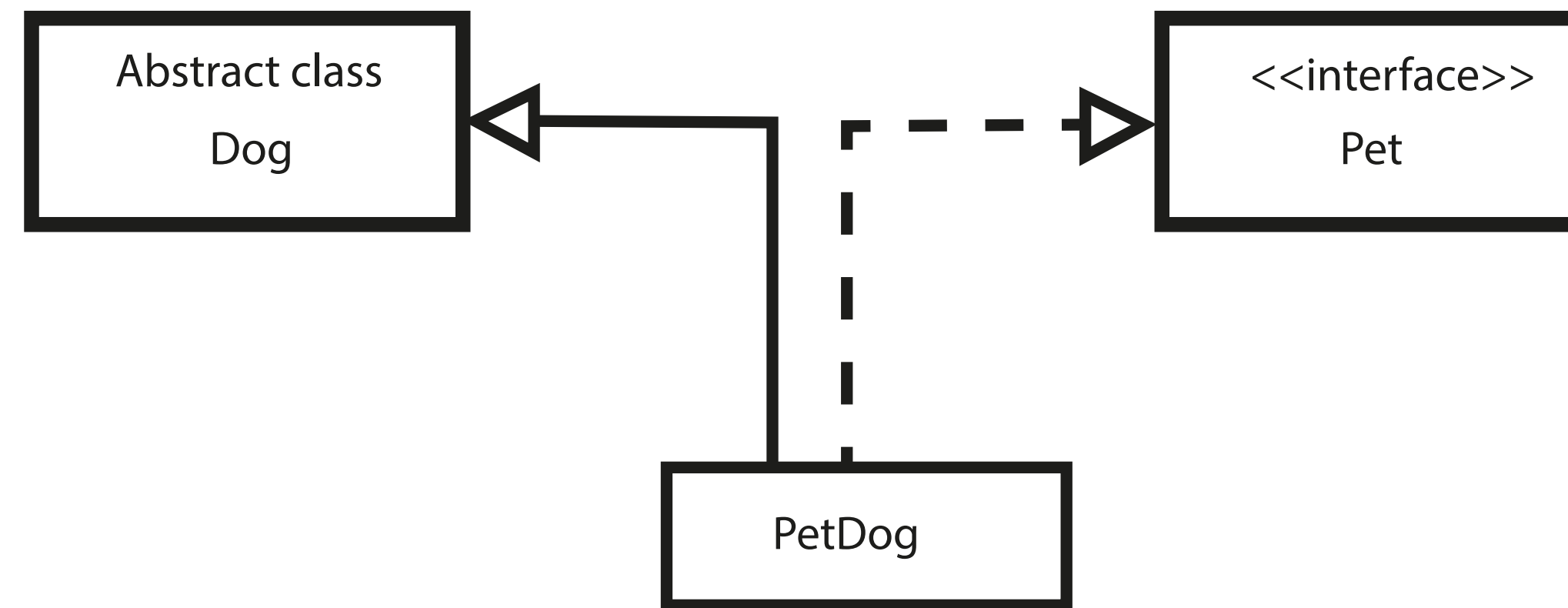
```
// The only file in the Arduino build that includes <Wire.h>.
```

Same contract: platform 2, Raspberry Pi

```
bool rpi::LinuxI2cBus::read(uint8_t reg, uint8_t* data, size_t len) {
    i2c_msg msgs[2] = {
        { address_, 0, 1, &reg }, // write register
        { address_, I2C_M_RD, uint16_t(len), data } // repeated start, read
    };
    i2c_rdwr_ioctl_data xfer{ msgs, 2 };
    return ::ioctl(fd_, I2C_RDWR, &xfer) >= 0;
}
```

```
// The only file in the Pi build that includes <linux/i2c-dev.h>.
// Bme280 did not change. Not one line.
```

Abstract classes vs. Interfaces: UML



Inheritance: PetDog is-a Dog (Realization)



Interface: PetDog implements Pet (Generalisation)



Implementing an Interface in C++

- See Demo Prog5_DogDemo_Interfaces.pdf (the classic example)
- This week the same idea on your own code: Bus is the interface, ArduinoI2cBus and LinuxI2cBus implement it, Bme280 only ever sees a Bus&.
- Code: Prog5/Code/week2 (core) and Prog5/Code/week4/rpi_platform (Pi).

Summary Implementing an Interface in C++

- A pure virtual function can *only be declared*, it *cannot be defined*.
- You cannot assign any value other than 0 to the pure virtual function.
- It is not possible to create an instance of a class with virtual functions: It would result in a compilation error.

Interfaces, what's all the fuzz about...

“No client should be forced to depend on methods it does not use.”

- Uncle Bob

<https://claudiorivera.net/2018/02/04/isp-interface-segregation-principle/>

Three ways to implement Interfaces in C++

Option 1: Multiple Inheritance

Option 2: Composition

Option 3: Interface Inheritance + Concrete Class

Option 4: (C++ ≥ 20) Templates and/or Curiously Recurring Template Pattern

Three ways to implement Interfaces in C++

Option 1: Multiple Inheritance

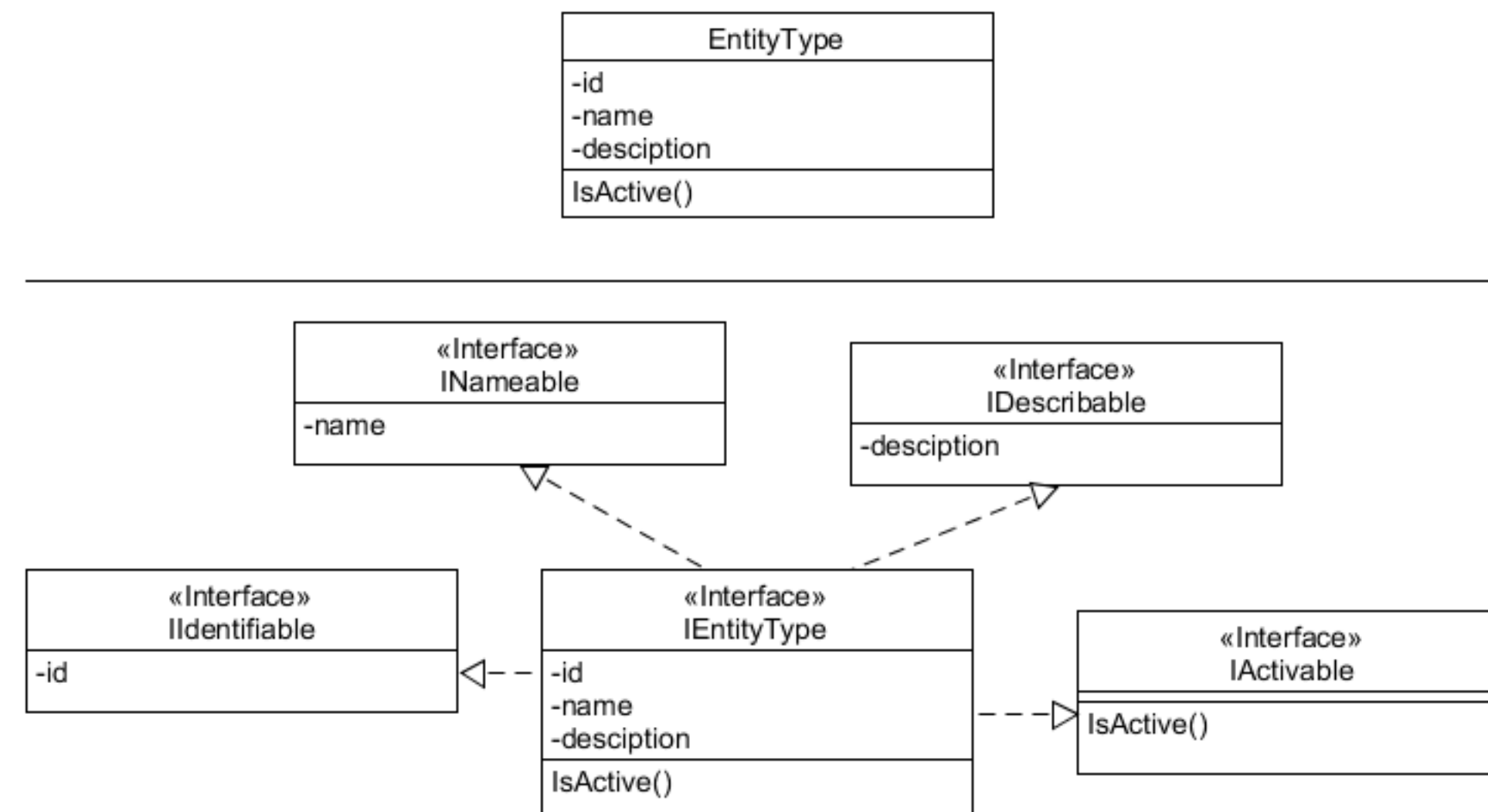
Option 2: Composition

Option 3: Interface Inheritance + Concrete Class

When considering SOLID principles, Option 3: Interface Inheritance + Concrete Class aligns best, especially with the Interface Segregation Principle (ISP) and the Dependency Inversion Principle (DIP).

- *Interface Segregation Principle (ISP)*
- *Dependency Inversion Principle (DIP) - Next week*

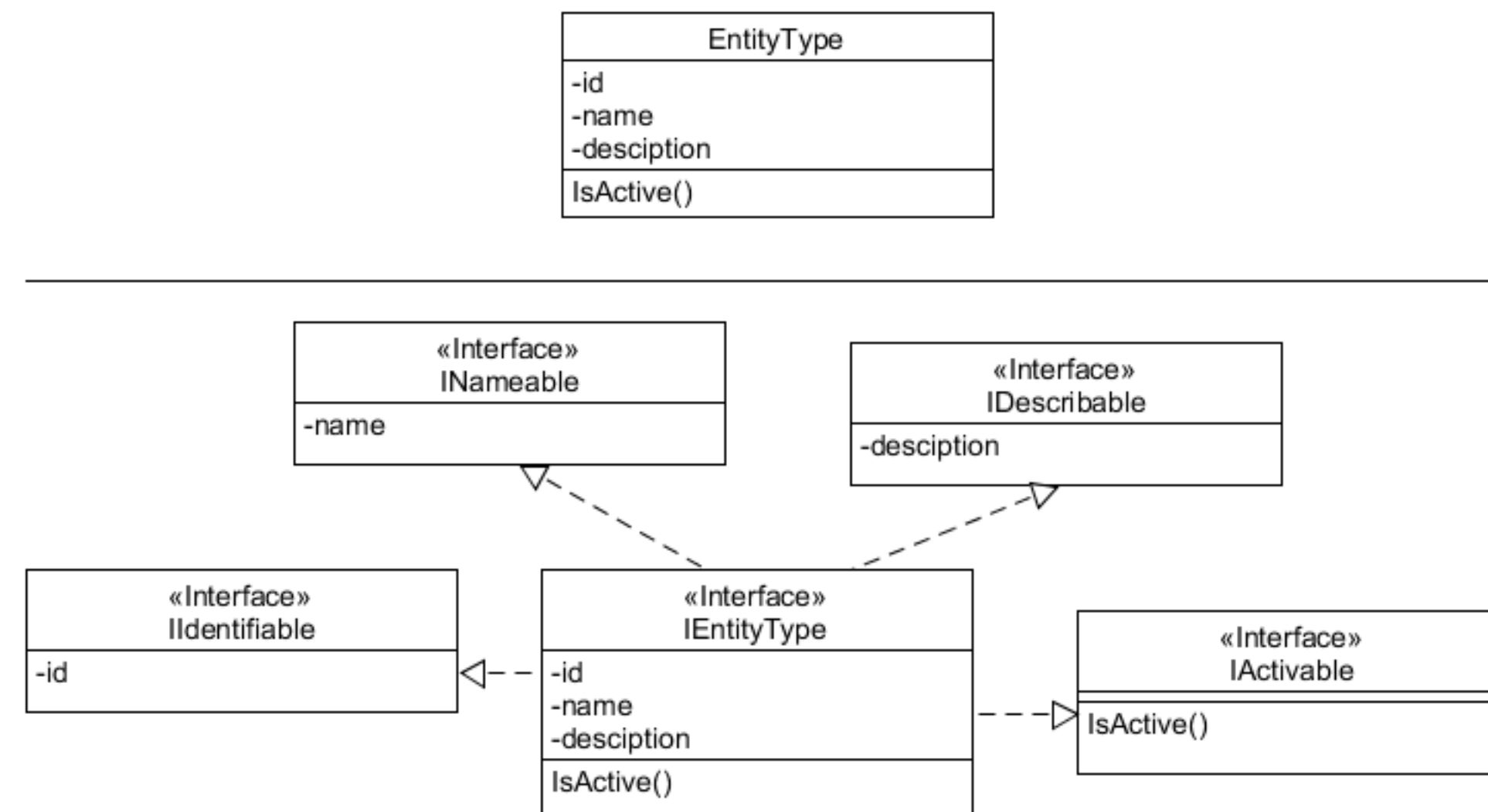
SOLID: Interface Segregation



<https://claudiorivera.net/2018/02/04/isp-interface-segregation-principle/>

SOLID: Interface Segregation

Make interfaces as slender as possible...



<https://claudiorivera.net/2018/02/04/isp-interface-segregation-principle/>

SOLID: Interface Segregation: summary

Clients should not be compelled to implement an interface that contains declarations of members or operations that they would not need or never use.

Interfaces that violate this principle are known as ‘fat’ or polluted interfaces as they contain too many operations (this violation is also known as ‘interface bloating’).

<http://stg-tud.github.io/sedc/Lecture/ws13-14/3.3-LSP.html#mode=document>

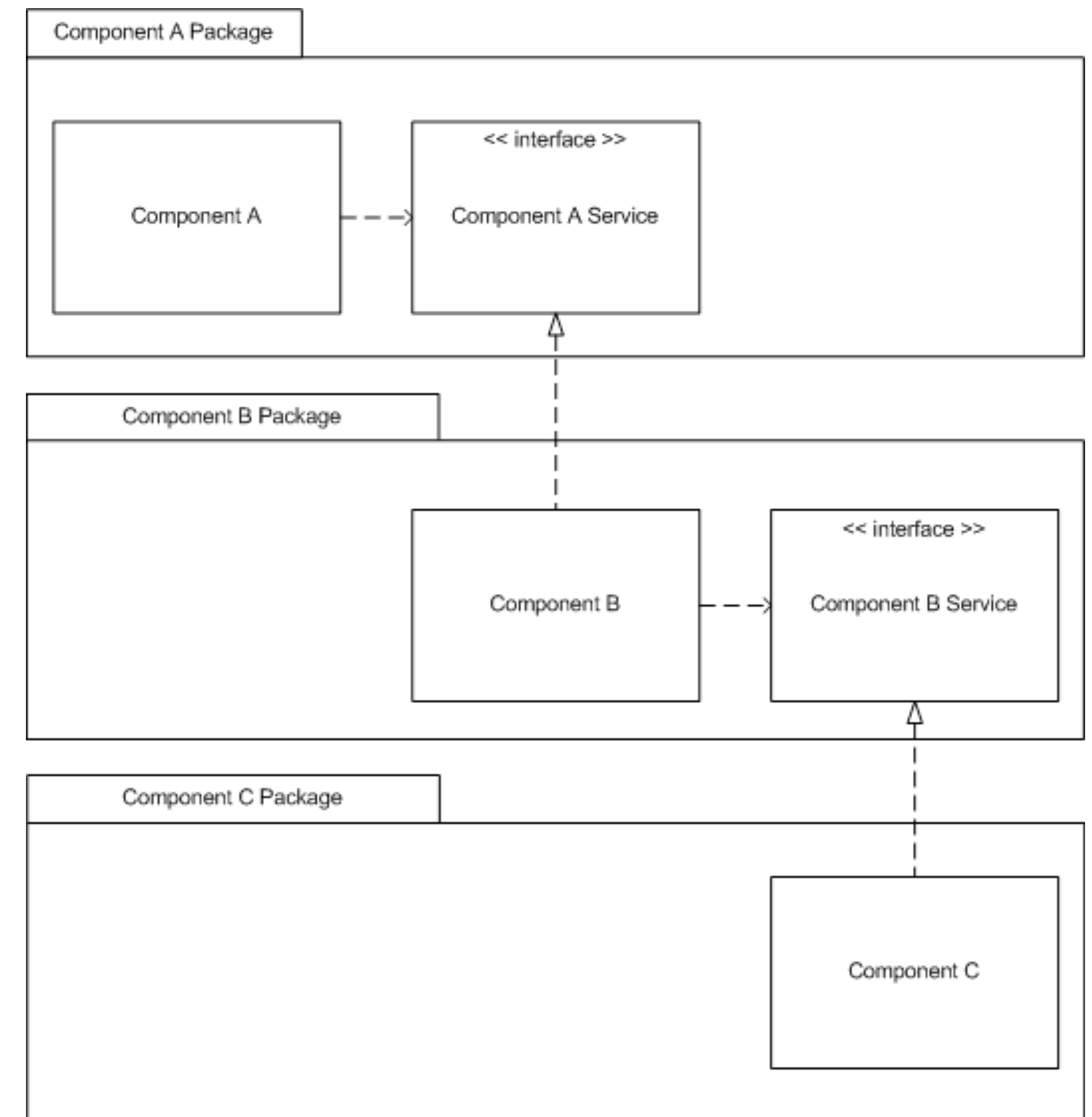
SOLID: Integration Segregation: summary

Helps to sustain low-coupling between the components comprising an application.

A. High-level modules should not depend upon low-level modules. Both should depend upon abstractions.

B. Abstractions should not depend upon details. Details should depend upon abstractions.

<https://dzone.com/articles/the-dependency-inversion-principle-for-beginners>



What to remember for CPP on Interfaces

- Define them as abstract classes
- How?

What to remember for CPP on Interfaces

- Define them as abstract classes
- Class is made abstract by declaring at least one of its functions as pure virtual function

What to remember for CPP on Interfaces

- Define them as abstract classes
- Class is made abstract by declaring at least one of its functions as pure virtual function
- A pure virtual function is specified by placing "= 0" in its declaration

Live demo: the same library on the Pi

On the Pi:

```
i2cdetect -y 1                # sensor at 0x76 or 0x77
cd Prog5/Code/week4/rpi_platform
cmake -B build && cmake --build build
./build/rpi_smoke_test
```

On the laptop:

```
cmake -B build && cmake --build build
-> core and tests build, rpi_smoke_test does not (Linux only)
```

That difference is the layering:

```
bmp280 (core) -> rpi_platform (Linux) -> rpi_smoke_test (app)
```

Interface Segregation, spotted

Look again at `Bus::delayUs()`.

A bus moves bytes. What does it have to do with time?

Who pays for it:

- MockBus in the tests must implement a delay it does not care about
- Arduinol2cBus and Linuxl2cBus copy the same three lines around different sleep calls
- a future SpiBus would copy them again

"No client should be forced to depend on methods it does not use."

Now the quote has a concrete victim.

The fix: Bus and Clock

```
class Bus {
public:
    virtual ~Bus() = default;
    virtual bool read(uint8_t reg, uint8_t* data, size_t len) = 0;
    virtual bool write(uint8_t reg, const uint8_t* data, size_t len) = 0;
};

class Clock {
public:
    virtual ~Clock() = default;
    virtual void delayUs(uint32_t us) = 0;
};

class Bme280 {
public:
    Bme280(Bus& bus, Clock& clock);    // two small contracts, not one fat one
    ...
};

// ArduinoClock, LinuxClock, FakeClock (tests: only counts the microseconds)
```

UML: dependencies. No arrow leaves the core

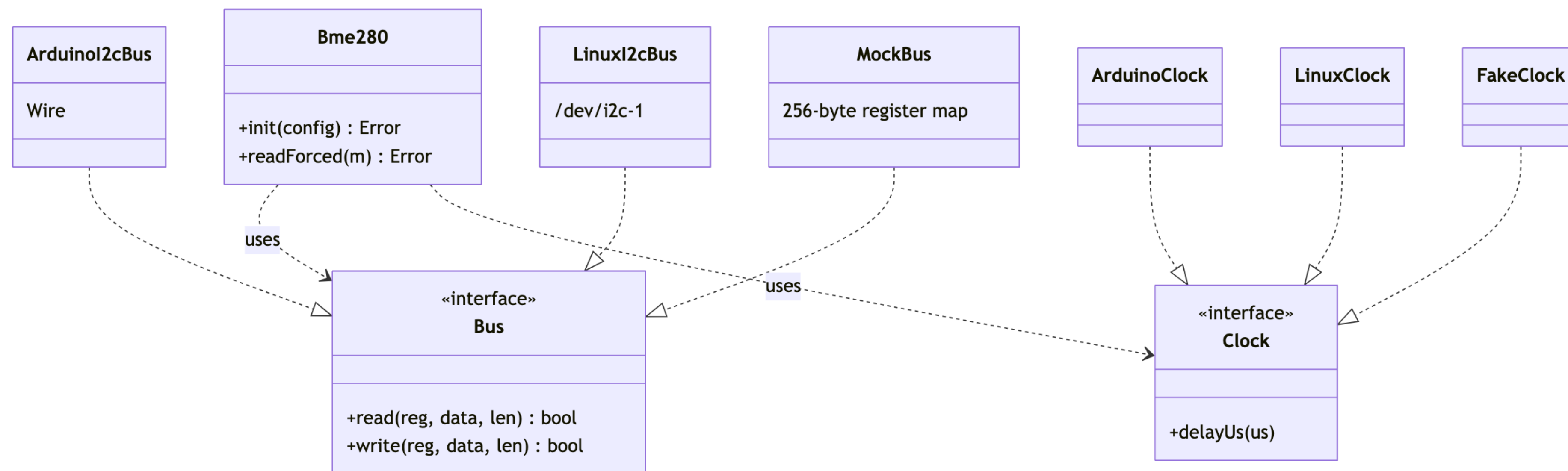
Realization (dashed, hollow head): implements

Dependency (dashed, open head): uses

Rule for your class diagram: not one arrow leaves the core towards a platform.

Try it: draw Wire and ioctl into this picture. They can only hang under the platform classes.

If you cannot draw it that way, the code has the problem, not the diagram.



Assignment 4: same library, second platform (deadline Thu 1 Oct, tag v0.4-pi)

1. Build the core on the Pi with CMake: three targets (Bosch C, wrapper, tests). Prove it builds on the Pi and on the laptop.
2. LinuxI2cBus: open in the constructor, close in the destructor, non-copyable.
3. Split Bus / Clock (ISP). ArduinoClock, LinuxClock, FakeClock. Write in the README what got simpler.
4. Smoke test on the Pi: one measurement per second, clean exit on Ctrl-C.
5. Class diagram with realizations and dependencies. No arrow leaves the core.

Layout: platform/arduino, platform/linux, examples/arduino, examples/rpi.

Full hand-out, starter code and tags: github.com/AEAEmbedded/ESE_PROG -> Prog5/Assignments

Links

- <https://www.codeproject.com/Articles/10553/Using-Interfaces-in-C>
- <https://data-flair.training/blogs/interfaces-in-cpp/>
- <https://www.tutorialcup.com/cplusplus/interfaces.htm>
- https://github.com/ksvbka/design_pattern_for_embedded_system/blob/master/design-patterns-for-embedded-systems-in-c-an-embedded-software-engineering-toolkit.pdf
- <https://martinfowler.com/articles/injection.html>

“

Any questions?

Try it yourself...

Cue: “What is the best way to use interfaces in C++?”

<https://copilot.microsoft.com/>

<https://gemini.google.com/>

<https://claude.ai/>

<https://chatgpt.com/>

Which one do you prefer?!